

# Análise dos Diferentes Procedimentos de Substituição de Memória Cache para Determinados Algoritmos

Guilherme S. L. Lopes – RA143630

<sup>1</sup>Universidade Estadual de Maringá – UEM

**Resumo.** *Este trabalho estuda como diferentes políticas de substituição afetam o desempenho de um computador. Foram realizados testes com o simulador gem5 utilizando seis políticas diferentes e três tipos de programas. Os resultados mostraram que a forma como o código é compilado torna a execução muito mais rápida, chegando a reduzir o tempo em até 85%, mesmo que isso diminua a taxa de acertos na cache. Além disso, percebeu-se que aumentar a quantidade de dados processados melhorou levemente a eficiência da cache.*

## 1. Introdução

A memória cache foi desenvolvida com o intuito de aumentar a eficiência de um computador, diminuindo a quantidade necessária de buscas na memória principal transferindo um bloco de memória por vez, ao invés de apenas uma única palavra [Stallings 2017].

Tendo isso em mente, foram-se desenvolvidas ao longo do tempo diversas formas de manipular a cache, visto que quando ela se enche, é necessário fazer uma troca nos blocos que estão sendo alocados nela, essas técnicas são chamadas de políticas de substituição. Como existem inúmeros diferentes algoritmos, nem todas as políticas são úteis, as vezes inclusive sendo realizado mais buscas que o necessário, fazendo com que a cache tenha seu um efeito contrário à seu propósito, por isso é importante avaliar quando uma política é útil e quando ela não é.

Pensando na efetividade das políticas de substituição, foi-se feita uma pesquisa para averiguar, qual é mais eficiente em programas de multiplicação de matrizes consecutivas, de cálculo de implementação de propagação de ondas e de cálculo de difusão de calor num espaço 3D – 3mm, ftd-2d e heat-3d respectivamente [Pouchet and Yuki 2016] – a fim de averiguar qual melhor se aplica a esses três.

## 2. Experimentos

### 2.1. Metodologia

Para esse estudo foi utilizado o simulador gem5 [Binkert et al. 2011], onde foi configurado uma máquina virtual com um processador single-core com velocidade de clock de 3GHz, uma RAM com frequência de 1600MHz e capacidade de memória de 8GB e, com níveis L1 e L2 da cache com tamanho de 32KiB.

Com essas configurações, foi feito testes usando o polybench nos algoritmos supracitados, usando de entrada mini e small datasets, com os tipos de compilação O0 e O3, sendo uma compilação padrão, sem otimizações, e uma compilação com a maior quantidade de otimização, respectivamente.

No experimento, serão usados 6 políticas de substituição: FIFO, LFU, LRU, MRU, RANDOM e WEIGHTED LRU. Dentro dessas, a única não vista em sala foi a

ultima citada, que consiste em uma variação da LRU padrão, onde é inserido pesos nos blocos a serem retirados, o com o menor peso é evitado e quando há dois com pesos iguais, evita-se o mais antigo [Luo et al. 2007].

## 2.2. Resultados

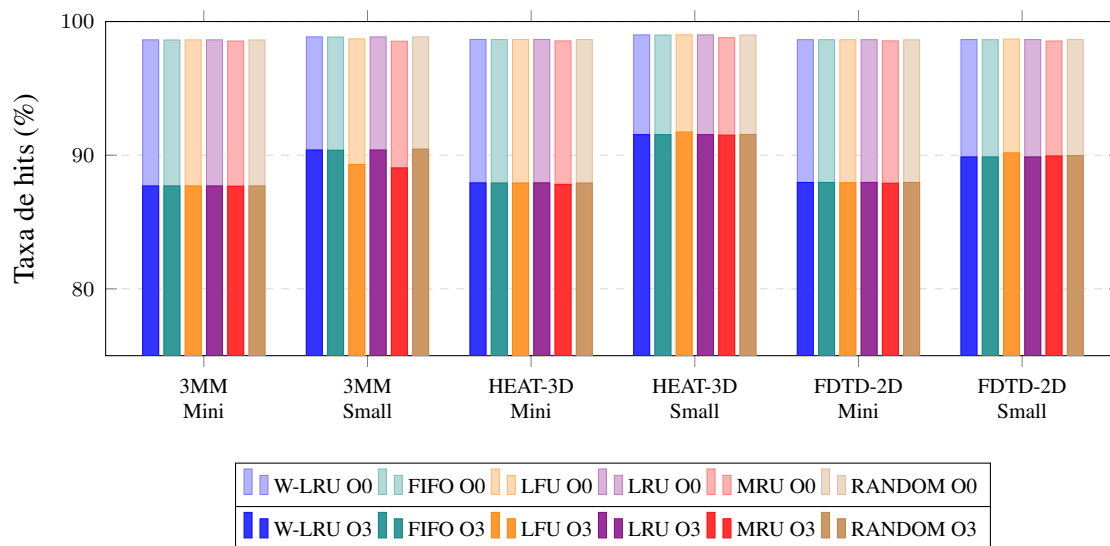


Figura 1. Taxa de hits por política

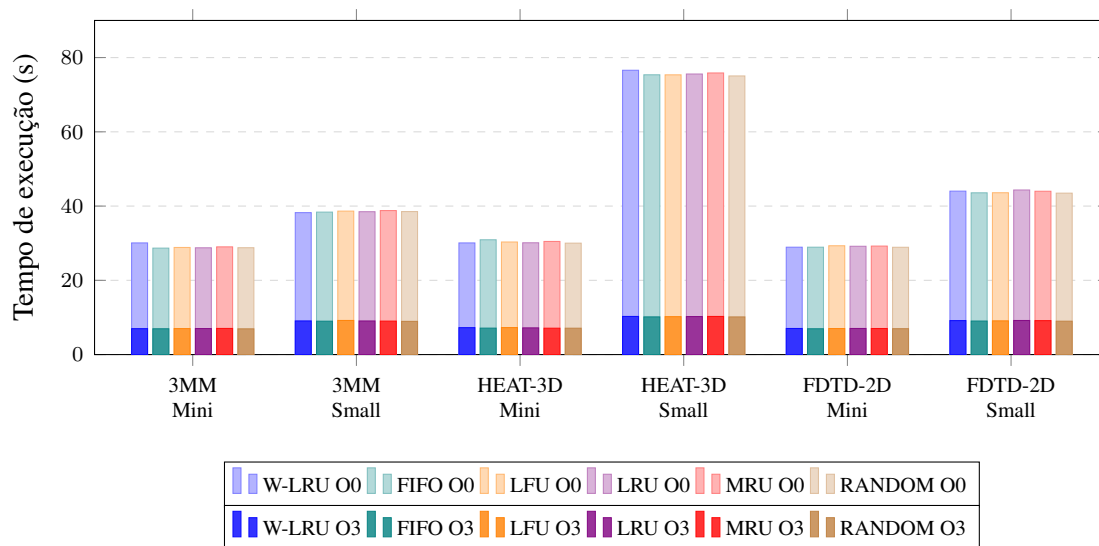


Figura 2. Tempo de execução por política

Como é possível observar na figura 1, usando o método de compilação com mais acertos de cache, foi justamente aquele com o menor nível de otimização, mantendo uma média de hits próxima dos 98%, porém também aumentou exponencialmente o tempo necessário para a execução, visível na figura 2.

É interessante de se notar especialmente que com a compilação O3 que, por mais que teve uma taxa de hits menor, então era de se esperar um tempo de execução maior,

devido a otimização na leitura dos algoritmos pelo compilador, o tempo de execução é no mínimo menor que a metade, e no caso do heat-3d pra um dataset small, é aproximadamente 15% do tempo de quando usamos a compilação O0.

Outra possível observação é que para o dataset de tamanho small, a taxa de hits – mesmo que não por muito – foi maior, aumentando em cerca de 2 à 3%, com 3 segundos a mais na execução em média. Como os algoritmos escolhidos são algoritmos mais complexos, com multiplicações contínuas de matrizes e cálculos espaciais, a expectativa era que o tempo de execução para o usando mais entradas seria obviamente maior, porém também é interessante notar que os hits também foram maiores, tendo aproximadamente um aumento de 12 milhões de hits, com 150 mil misses a mais, excluindo a MRU, que teve em todos um aumento expressivo de misses, alcançando 300 mil a mais na média.

### 3. Conclusão

Analisando os dados, podemos ver que a taxa de hits na compilação O0 é melhor em todos os algoritmos, porém o elevado tempo de execução torna inviável dependendo da quantidade de vezes que é necessária rodá-lo. Um possível cenário onde possa ser mais interessante o uso desse modo, é com algoritmos menores e menos complexos, visto que o tempo necessário reduziria e os acertos da cache se manteriam elevados.

Já para a compilação O3, mesmo mantendo uma quantidade menores de acertos, o tempo de execução se reduziu tanto a ponto de talvez ser mais viável utilizar esse método para algoritmos complexos, sacrificando a taxa de hits para ganhar mais execuções. Possivelmente, com um dataset maior ainda os acertos podem melhorar, uma vez que teve um aumento no dataset mini para o dataset small, é possível considerar uma outra pesquisa para verificar se com datasets maiores ainda, essa ideia ainda é válida.

### Referências

- Binkert, N. L., Beckmann, B. M., Black, G., Reinhardt, S. K., Saidi, A. G., Basu, A., Hestness, J., Hower, D., Krishna, T., Sardashti, S., Sen, R., Sewell, K., Altaf, M. S. B., Vaish, N., Hill, M. D., and Wood, D. A. (2011). The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7.
- Luo, Y., Changsheng, X., and Chengfeng, Z. (2007). Weighted cache replace algorithm for storage system. *International Journal of Computational Intelligence Systems*.
- Pouchet, L.-N. and Yuki, T. (2016). PolyBench/C: The polyhedral benchmark suite. <https://www.cs.colostate.edu/~pouchet/software/polybench/>. Version 4.2.1.
- Stallings, W. (2017). *Arquitetura e Organização de Computadores*. Pearson Education do Brasil, São Paulo, 10 edition.